

Final Report: Privacy-First Email Prioritization with Active Learning on Small Language Models

Hansheng Zeng

Jiaming Zhao

JJ Sandhu

Abstract

We study a privacy-first email prioritization system built around small language models (SLMs) that run locally. Our pipeline assigns structured urgency signals per email, refines predictions via clustering-based joint reasoning, and adapts to individual users through an active learning loop. We establish pilot baselines on 100 manually labeled Enron emails, comparing a keyword heuristic, a locally served Qwen2.5-1.5B, and cloud GPT-5.4. The cloud model currently leads on urgency ranking (Spearman $\rho=0.615$), while the local SLM already meets latency targets (<600 ms), suggesting the main challenge is calibration and personalization rather than inference speed. Building on this diagnosis, our final system adds (i) a 13-feature post-hoc calibrator that lifts the SLM’s macro-F1 from 0.28 to 0.39 (+39% relative) without re-querying the model, (ii) a calibrated few-shot prompt with an explicit class prior and a continuous urgency score, and (iii) an interactive active-learning CLI whose prompt-manager rewrites the empirical prior and rotates class-balanced demonstrations. We surface a counterintuitive empirical result: random sampling matches or slightly exceeds entropy-based AL at every label budget on this dataset—an artefact of the SLM’s three-bin confidence string. The full pipeline runs in ~ 58 s for 100 emails on CPU and ships zero email content off-device.

1 Introduction

Large language models have rapidly become the default interface for inbox triage in commercial mail clients, where they summarize threads, draft replies, and flag action items (Microsoft, 2025). These deployments rely almost universally on remote inference: the user’s full message bodies are transmitted to a service-provider endpoint and processed on shared infrastructure. This design choice is at odds with two emerging realities. First, regulated and security-sensitive users—healthcare, legal, and enterprise compliance—are increasingly prohibited from exporting raw email content beyond the device. Second, recent disclosures of cross-prompt injection attacks against cloud email summarizers (Ahmeti and Permiso Security, 2026) demonstrate that a remote model with read access to every message is itself an attack surface, with adversarial content in a single email capable of hijacking assistant behavior across an entire inbox.

A natural alternative is to perform inference entirely on-device. Recent work on small language models in the 1–3B parameter regime has narrowed the gap to cloud LLMs on instruction-following tasks, and post-training quantization (Frantar et al., 2023; Lin et al., 2024) together with parameter-efficient adaptation (Hu et al., 2022; Dettmers et al., 2023) has made commodity-hardware deployment practical (Lu et al., 2024). Apple Intelligence demonstrates that an on-device ~ 3 B model coupled with task-specific adapters can reach product scale (Apple Machine Learning Research, 2024). However, naively replacing a cloud LLM with an SLM does not yield a workable prioritization system, for three reasons that we observe directly in our pilot study (§3). (i) Per-email zero-shot inference cannot resolve urgency that depends on cross-message context, such as deadlines implicit in earlier threads or shared recipients. (ii) Quantized SLMs are systematically miscalibrated on this task, over-predicting urgency by roughly $4\times$ on real inboxes. (iii) What constitutes an “urgent” message is highly user-dependent, but the standard remedies—fine-tuning, online RLHF, or cloud-side preference learning—all violate the on-device constraint.

In this paper, we propose a three-stage local pipeline that addresses these failure modes without leaving the device. Stage 1 prompts a quantized SLM to emit *structured* per-email outputs (urgency tier, deadline estimate, calibrated confidence, missing-context flag) rather than a single label. Stage 2 groups related emails by thread structure, sender overlap, and semantic similarity, and performs cluster-level joint refinement that reconciles per-email predictions and surfaces context gaps invisible to a single-message view. Stage 3 closes the loop with an entropy-driven active learning step: the highest-uncertainty predictions are surfaced to the user, and corrections are distilled into prompt updates that personalize the SLM without modifying its weights.

Empirical findings. On 100 manually labeled emails from the Enron corpus, the local SLM meets the latency target (587 ms vs. a 2 s budget) but trails a cloud upper bound on rank correlation (Spearman $\rho=0.24$ vs. 0.62). A keyword heuristic obtains higher *accuracy* than the SLM by exploiting class imbalance, but achieves zero recall on the Normal tier—a reminder that accuracy is misleading on long-tailed inboxes. Together, these results localize the bottleneck to calibration and personalization rather than raw inference cost, motivating the cluster-level refinement and active learning components of our pipeline.

Contributions.

- **Privacy-first email prioritization as a structured prediction task.** We replace the conventional binary importance setting with a multi-output formulation (label, deadline, confidence, missing-context flag) suited to ranking and context-gap detection under strict on-device constraints.
- **A two-stage local SLM pipeline** that decouples per-email inference from cluster-level joint refinement, enabling cross-message reasoning while preserving locality.
- **Entropy-based on-device personalization** via prompt updates rather than weight updates, allowing user feedback to shape the model without leaving the device.
- **Pilot baselines on real-world email** that quantify the gap between heuristic, on-device SLM, and cloud LLM prioritization, and isolate the calibration–personalization bottleneck targeted by the full system.

2 Related Work

Email prioritization. Inbox triage was framed as a learning problem by Whittaker and Sidner (1996), who characterized email as overloaded with task and reminder content beyond pure communication, and by Dabbish et al. (2005), who modeled per-message action probability from user behavior. The largest-scale instantiation is Gmail’s Priority Inbox, which fits per-user logistic models on implicit interaction signals (Aberdeen et al., 2010). The Enron corpus has served as the standard benchmark for downstream tasks including folder prediction and importance estimation (Klimt and Yang, 2004; Bekkerman et al., 2005). These approaches share two assumptions we explicitly relax: a centralized server learns user-specific parameters from logged behavior, and the prediction target is a single scalar action probability. We instead operate on-device with no behavioral logging, and predict a structured tuple (label, deadline, confidence, context flag) that supports downstream cluster-level reasoning.

LLM-based email assistants. Microsoft 365 Copilot integrates cloud LLMs into mail clients within an enterprise tenant boundary, but inference is remote and message bodies leave the device (Microsoft, 2025). Cross-prompt injection attacks against such pipelines (Ahmeti and Permiso Security, 2026) have demonstrated that this exposure is exploitable in practice. At the opposite end of the design space, Apple Intelligence runs a $\sim 3\text{B}$ -parameter foundation model directly on Apple silicon with task-specific LoRA adapters and falls back to a Private Cloud Compute server only for queries the on-device model cannot handle (Apple Machine Learning Research, 2024). Our setting is the on-device end, but without privileged access to a hardware-accelerated foundation model: we restrict ourselves to an open SLM served by a commodity local runtime, and ask how far structured prompting and clustering-based refinement can push such a system.

On-device small language models. Recent SLMs in the 1–5B range achieve competitive instruction-following performance on consumer hardware (Qwen Team, 2024; Lu et al., 2024). Two compression directions enable this regime: post-training quantization, with activation-aware (AWQ, Lin et al., 2024) and reconstruction-based (GPTQ, Frantar et al., 2023) variants reducing weights to 4–8 bits; and parameter-efficient adaptation via LoRA (Hu et al., 2022) and QLoRA (Dettmers et al., 2023) that enables per-user fine-tuning on a single consumer GPU. Lu et al. (2024) report that runtime cost—prefill, decode, and energy—dominates parameter count in determining edge feasibility, consistent with our pilot finding that a 1.5B-parameter quantized model is fast enough but miscalibrated. We use these techniques as building blocks rather than contributions; the novel component is the structured prompting interface that exposes calibrated confidence for downstream use.

Active learning. The standard pool-based formulation (Settles, 2009) ranks unlabeled instances by an acquisition function and queries the most informative ones. For probabilistic classifiers, the two dominant choices are predictive entropy and Bayesian Active Learning by Disagreement (BALD, Hounsby et al., 2011), which targets points of high epistemic uncertainty by measuring disagreement among posterior samples; deep variants typically realize the posterior

via Monte Carlo dropout or model ensembles (Gal et al., 2017), and Ren et al. (2021) survey the broader design space. Both BALD-style estimators conflict with single-model on-device deployment of a quantized SLM, since they require either multiple stochastic forward passes or multiple model copies. We therefore use predictive entropy computed directly on the calibrated softmax output of Stage 1, which adds no additional inference cost and is consistent with Ren et al. (2021)’s finding that entropy is a strong baseline at small label budgets.

Prompt-based adaptation. Because we adapt the SLM through prompt edits rather than weight updates, the closest line of work is automatic prompt optimization. Zhou et al. (2023) introduce APE, which generates and selects natural-language instructions from input–output demonstrations using a powerful LLM as the proposal distribution. Pryzant et al. (2023) propose ProTeGi, an iterative procedure that edits prompts in directions implied by natural-language critiques of model errors. Both methods assume a frontier LLM serves as the editor and operate on public benchmarks. Our personalization loop runs the same SLM in both predictor and editor roles, on a small user-specific correction set surfaced by active learning, trading exploration breadth for end-to-end privacy and on-device feasibility. Our clustering refinement uses Sentence-BERT-style bi-encoders (Reimers and Gurevych, 2019) and HDBSCAN (McInnes et al., 2017) as off-the-shelf components.

3 Pilot Study

Before describing the full system, we report a pilot study that measures the gap between three reference points—a heuristic, an on-device SLM, and a cloud LLM—on a small, manually labelled subset of real email. The findings of this pilot directly motivate the design choices in §4.

3.1 Dataset and Metrics

We evaluate on 100 emails from the inbox of Enron user `arnold-j` (Klimt and Yang, 2004), each parsed into subject, sender, date, and body (truncated to 1,000 characters). Emails are manually labeled with three urgency tiers: **Urgent** (response within 24h), **Normal** (within 3 days), and **Low** (no time pressure). The distribution is skewed: 76 Low, 13 Normal, 11 Urgent.

We report three metrics. **Spearman** ρ measures monotonic association between predicted and ground-truth urgency rankings (Low=1, Normal=2, Urgent=3). **Accuracy** measures exact label match; given the class imbalance we also report **macro-F1**, which weights all three classes equally. We further define **missing-context precision** ($|M_{\text{pred}} \cap M_{\text{gold}}|/|M_{\text{pred}}|$) as a target metric for the full system, evaluated separately in §5.

3.2 Pilot Baselines

We compare three reference points: (1) a **keyword heuristic** scanning for urgency cues (e.g., “asap,” “deadline”), (2) **Qwen2.5-1.5B** (Qwen Team, 2024) served locally via Ollama (Ollama, 2026) with a structured zero-shot prompt, and (3) **GPT-5.4** (OpenAI, 2026) as a cloud upper bound using the same prompt. The full evaluation in §6 stacks our calibrators on top of (2); the cloud upper bound (3) is held fixed throughout.

Table 1: Pilot baseline comparison on 100 Enron emails.

Method	Spearman ρ	p -value	Accuracy	Avg. Latency
Heuristic (Keyword)	0.318	0.001	0.680	<1 ms
SLM (Qwen2.5-1.5B)	0.240	0.016	0.280	587 ms
GPT-5.4 (Cloud)	0.615	<0.001	0.710	4,993 ms

Table 2: Per-class recall and predicted label distribution (ground truth: 11/13/76).

Method	Recall			Predicted Count		
	Urg.	Norm.	Low	Urg.	Norm.	Low
Heuristic	0.636	0.000	0.803	18	8	74
SLM	0.727	0.538	0.171	40	46	14
GPT-5.4	0.455	0.846	0.724	12	33	55

3.3 Findings That Motivate the Pipeline

Tables 1 and 2 expose four patterns that drive the design of §4.

(F1) GPT-5.4 leads, but the gap is in ranking, not classification. With Spearman $\rho=0.615$ and accuracy of 71%, the cloud model ranks emails most effectively. Its predicted distribution (12/33/55) is closest to ground truth (11/13/76), though it still over-predicts Normal at the expense of Low. The accuracy gap to the keyword heuristic is small (3 pp); the ranking gap is large ($\Delta\rho = 0.30$). This drives our choice of macro-F1 and Spearman ρ , rather than accuracy, as the headline metrics in §6.

(F2) The keyword heuristic looks good on accuracy because the inbox is long-tailed. Its 68% accuracy is largely driven by the skewed class distribution: by defaulting most emails to Low, it captures 80.3% of the majority class while completely failing to identify Normal emails (0% recall). Accuracy on imbalanced inboxes is misleading—a non-trivial baseline gets 68% by doing essentially nothing.

(F3) The on-device SLM substantially over-predicts urgency. It labels 40 of 100 emails as Urgent (vs. 11 in ground truth) and achieves only 17.1% recall on Low-urgency emails. Calibration, not better prompting alone, is needed to match the ground-truth distribution. This finding directly motivates the dual refinement of Stage 1 in §4.1: a calibrated few-shot prompt to attack the bias at decoding time, plus a feature-based post-hoc calibrator to correct the residual.

(F4) Latency is not the bottleneck. The SLM averages 587 ms per email, well within the 2 s target. The cloud upper bound takes $\sim 8.5\times$ longer per call. The bottleneck for an on-device system is therefore prediction quality, not raw inference cost—which justifies spending the entire §4 budget on calibration, cross-message reasoning, and personalization rather than on inference acceleration.

4 Method

We organize the description by the order in which an email traverses the pipeline. §4.1 covers per-email structured prediction with the SLM, including both the calibrated few-shot prompt and a 13-feature post-hoc calibrator that refine the base output without re-querying the model. §4.2 re-scores predictions using cross-email context. §4.3 closes the loop with entropy-driven active learning, both in simulation and as a real interactive command-line tool with a prompt manager that personalizes the SLM through prompt edits rather than weight updates. The complementary missing-context detection module is described separately in §5.

4.1 Stage 1: Local Urgency Inference

Each email is scored by a quantized SLM on CPU. The base prompt provides the model with the email subject, sender, and body (truncated to 800 tokens) and asks for a JSON object with an urgency label $u \in \{\text{urgent}, \text{normal}, \text{low}\}$, deadline estimate, confidence level, and a missing-context flag. As reported in §3.3 (F3), this base configuration is severely mis-calibrated on the pilot, predicting 40 `urgent` labels against a ground-truth count of 11. We refine the per-email prediction along two complementary axes: a redesigned prompt that addresses miscalibration at decoding time, and a feature-based post-hoc calibrator that addresses the residual after decoding. The two refinements are independent and stack additively.

Calibrated few-shot prompt. We re-engineered the SLM prompt with three changes. **(1) Explicit prior in the system prompt:** the system message states the empirical class distribution—“Most emails are LOW. In a typical inbox, $\sim 75\%$ are LOW, $\sim 15\%$ NORMAL, $\sim 10\%$ URGENT.”—which the SLM reliably attends to, reducing the urgent rate from 40% to $\sim 25\%$ by itself. **(2) Five class-balanced demonstrations** sampled to match the empirical 1:1:8 ratio (one urgent, one normal, three low), covering the failure modes the bare prompt got wrong: vendor newsletters, FYI forwards, soft “could you” requests, ambiguous-deadline coordination, and an explicit-EOD prototype as the urgent demo. **(3) Continuous urgency_score (0–100) and signals list:** the output schema gains a continuous score that downstream ranking uses; the discrete label becomes `argmax` over the score. The 3-bin confidence string is removed—it had been collapsing 100 emails onto only nine (label, conf) buckets, which directly caused the entropy-AL failure documented in §7. The `signals` list (e.g., `["explicit_deadline", "action_request"]`) is consumed by the cluster-refinement and missing-context modules. The full prompt fits in ≈ 600 input tokens; one decode is ≤ 220 output tokens.

Post-hoc calibration. The few-shot prompt removes most of the global over-prediction bias but leaves a feature-dependent residual—e.g., vendor newsletters whose boilerplate happens to contain the word “deadline”—that a fixed prompt cannot exploit. We add a feature-based post-hoc calibrator that fits on the user’s labelled data and runs entirely on-device. We deliberately did not pursue fine-tuning: CPU fine-tuning of a 1.5B model is too slow for an interactive personalisation loop, and parameter-efficient methods like LoRA still require a GPU we cannot assume on target hardware; cloud fine-tunes would defeat the privacy story.

For each email we compute a 13-dimensional vector with no additional LLM calls: (i) one-hot SLM label (3 dims); (ii) confidence string mapped to a scalar (`high`=0.85, `medium`=0.6, `low`=0.4); (iii) the SLM’s missing-context flag; (iv) log-scaled body and subject lengths (2 dims); (v) lexical counts of urgent and softening keywords (2 dims); (vi) markers for forwarded thread and URL presence (2 dims); and (vii) sender-domain buckets (internal Enron, government, `no-reply` prefix; 3 dims combined). All features are min–max scaled per fold. We compare six predictors with stratified 5-fold CV: prior shift (Saerens et al., 2002) (closed-form re-weighting using only the empirical prior), logistic regression with and without `class_weight=balanced`, random forest (300 trees, balanced class weights), gradient boosting (200 trees, depth 3, learning rate 0.05), and ridge rank regression (Ridge against urgency *rank* 1/2/3, rounded to nearest discrete label). Class-balanced LogReg becomes our canonical calibrator throughout the rest of the report; the others are reported in Table 3.

4.2 Stage 2: Clustering Refinement

Per-email scores are refined using cross-email context. Emails are encoded into 384-d embeddings, grouped via density-based clustering, and each email’s urgency is updated as:

$$\hat{u}_i = (1 - \alpha\gamma_c) u_i + \alpha\gamma_c \bar{u}_c \quad (1)$$

where $\bar{u}_c$ is the cluster mean urgency, $\alpha=0.3$ is the refinement weight, and $\gamma_c = \max(0, 1 - \sigma_c/2)$ down-weights high-dispersion clusters.

4.3 Stage 3: Active Learning

Stage 3 closes the loop by surfacing high-uncertainty predictions to the user, then folding the corrections back into the SLM prompt. We describe the simulated entropy-based selection rule first, then the interactive command-line tool and prompt manager that deploy the loop on a real user.

Entropy-based selection. We convert SLM confidence into pseudo-probabilities and compute Shannon entropy $H_i = -\sum_c p_{ic} \log p_{ic}$, selecting the top- B most uncertain emails for user correction. In simulation, entropy-based active learning improves SLM accuracy from 28% to 64% with 50 labels and Spearman ρ from 0.24 to 0.44.

Interactive CLI and prompt manager. The midpoint AL loop was simulated against ground truth. The final pipeline includes an interactive command-line tool plus a prompt-manager module that together close the personalisation loop on a real user. The CLI ranks unlabelled emails by entropy (with optional cluster-diversity round-robin), presents the top batch one email at a time, and accepts a single keystroke from {u, n, l, s, q} (urgent / normal / low / skip / quit)

plus an optional free-text note. Each correction becomes an event written to an on-disk `prompt_history.json` that persists across sessions.

The prompt manager rebuilds the SLM prompt for the next iteration with two history-compression mechanisms keeping the prompt under $\sim 1.5k$ tokens regardless of correction count. **(a) Prior summary:** the system prompt’s prior sentence is regenerated from the user’s empirical class distribution after every batch (e.g., “Based on 50 emails this user has labelled, the empirical distribution is 82% LOW, 12% NORMAL, 6% URGENT.”). **(b) Class-balanced demo retention:** the demo pool is capped at 8; retention keeps one demo per class first, then fills with most-recent corrections regardless of class. The class floor prevents the demo pool from drifting majority-ward as the user labels more, which would otherwise reintroduce the over-prediction problem. **Re-scoring policy:** we re-query the SLM only when (i) the user explicitly requests it or (ii) the running prior summary diverges from the previous one by >5 pp on any class; on a 50-correction oracle replay this triggers re-scoring twice (after the 20th and 40th correction).

5 Missing-Context Detection

A useful triage system should flag not only the predicted urgency of a message but also whether that prediction was made on incomplete information. We define an email as having *missing context* when a confident urgency judgment requires evidence that is not in the message body itself—for example, an unresolved deadline referred to as “ASAP,” a reply that begins “as we discussed,” or a forwarded thread whose head is missing. This module operates in parallel to the urgency-prediction pipeline of §4: rather than altering the predicted label, it surfaces a separate signal that the user can use to defer or escalate decisions.

5.1 Detector Design

We implement five lightweight regex-based detectors targeting recurring missing-context patterns observed in pilot annotation: (i) missing-attachment cues (“see attached,” “PFA,” “enclosed”), (ii) forwarded-thread markers, (iii) references to prior conversation (“as we discussed,” “per our call”), (iv) ambiguous deadlines (“ASAP,” “EOW,” “at your earliest convenience”), and (v) external links not on a small whitelist of standard meeting platforms. Each detector contributes a confidence in $[0, 1]$ (0.85, 0.75, 0.70, 0.65, 0.50 respectively, calibrated by inspection on a held-out sample), and an email is flagged if any detector fires above its threshold.

5.2 Proxy Evaluation and Its Limits

A direct evaluation of missing-context precision requires per-email human annotation, which is expensive at the dataset’s scale. We first attempted a *proxy oracle*: an email is treated as a missing-context positive if the ground-truth label is `urgent` but the SLM’s prediction is `low`, on the assumption that such cases reflect missing evidence rather than annotation noise. On the 100-email pilot the proxy yields only a single positive, so the reported numbers (heuristic detectors flag 42 / 100 emails; the SLM’s built-in flag fires on 3 / 100) function as a lower bound on coverage rather than a precision estimate.

5.3 Human-Annotated Evaluation

To obtain a usable signal, we extended the active-learning loop (§4.3) to collect, for 35 emails surfaced over 7 iterations of batch size 5, both an urgency correction and a binary missing-context judgment with a free-text rationale. Two findings emerge.

- **The SLM substantially under-flags missing context.** Human annotators flagged 19 of 35 emails (54.3%) as missing context; the SLM’s built-in flag fires on 3 of 35 (8.6%), giving a recall of $3/19 = 15.8\%$ against the human oracle. The proxy estimate on the full pilot understated this gap.
- **The SLM and humans flag different things.** Categorizing the 19 human-flagged cases yields three recurring failure modes: *unfamiliar relational context* (10 cases, e.g., “who is Kim?”, “who, when?”), *absent prior thread or background* (8 cases, e.g., “previous email”, “importance of the meeting”), and *message bodies too short to interpret* (4 cases, e.g., “everything is missing”). The SLM’s own flag, in contrast, fires almost exclusively on short or atypically formatted machine-generated emails. The model conflates “unusual surface form” with “insufficient evidence,” and is blind to the relational gaps that humans actually experience.

The detector outputs are surfaced in the user interface as concrete evidence spans (e.g., the highlighted phrase “as we discussed last week”) rather than a binary flag, so that a user can sanity-check the system’s reasoning at the point of decision. We treat the human-annotated numbers as a calibration target: closing the 15.8% recall gap is the primary objective for the post-midpoint phase.

6 Full Evaluation

Setup. We re-use the dataset, label scheme, and metrics defined in §3.1. We evaluate the seven systems summarized in Table 3: the keyword heuristic, the raw SLM, the five post-hoc calibrators stacked on the few-shot SLM (§4.1), and GPT-5.4 as a cloud upper bound. Beyond the pilot reference points (rows 1, 2, 8), the new rows in Table 3 compare calibrator backends, all evaluated on the same 100 emails with stratified 5-fold CV.

Hyperparameters. Qwen2.5-1.5B is served via Ollama with temperature=0, top- $p = 1.0$, max_tokens=220, and JSON mode enabled. SBERT embeddings use all-MiniLM-L6-v2 with normalisation. HDBSCAN uses min_cluster_size = 4 and min_samples = 2 with default (Euclidean) distance. Calibrators use scikit-learn defaults except as noted in §4.1. Active learning runs with 5 random seeds for the random arm and deterministic ordering for the entropy arms; budgets are {0, 5, 10, 20, 30, 50, 75} corrections.

Table 3: Main results, 5-fold stratified CV on the 100-email pilot. Class-balanced LogReg lifts macro-F1 **0.28** $\rightarrow$ **0.39** (+39% relative) over the raw SLM while preserving Spearman ρ . Calibrators that exploit the 76% low prior achieve higher accuracy by collapsing predictions toward low (predicted distributions {6, 1, 93}, {5, 3, 92}, {5, 10, 85}).

Method	Acc	macro-F1	ρ	Rec. Urg.	Rec. Norm.	Rec. Low	U / N / L
Keyword heuristic	0.68	0.43	0.32	0.64	0.00	0.80	18 / 8 / 74
SLM raw (Qwen2.5-1.5B)	0.28	0.28	0.24	0.73	0.54	0.17	40 / 46 / 14
+ Prior shift (Saerens et al., 2002)	0.68	0.34	0.07	0.00	0.23	0.86	0 / 16 / 84
+ LogReg (balanced) [ours]	0.43	0.39	0.26	0.64	0.46	0.40	23 / 40 / 37
+ LogReg (unbalanced)	0.76	0.41	0.23	0.27	0.00	0.96	6 / 1 / 93
+ Random forest	0.72	0.36	0.25	0.18	0.00	0.92	5 / 3 / 92
+ Gradient boosting	0.70	0.41	0.17	0.18	0.15	0.87	5 / 10 / 85
GPT-5.4 (cloud, upper bound)	0.71	0.58	0.62	0.46	0.85	0.72	12 / 33 / 55

Table 4: Feedback efficiency: accuracy and Spearman ρ as the user labels increasing fractions of the inbox. Random sampling is averaged over five random seeds; entropy and entropy + cluster-diverse use deterministic ordering. Random is competitive at all budgets and slightly better on ρ .

Labels	Random Acc	Entropy Acc	Diverse Acc	Random ρ	Entropy ρ	Diverse ρ
0	0.28	0.28	0.28	0.24	0.24	0.24
10	0.35 $\pm$ 0.02	0.34	0.31	0.29	0.26	0.27
20	0.43 $\pm$ 0.02	0.43	0.38	0.36	0.30	0.33
30	0.50 $\pm$ 0.02	0.51	0.43	0.40	0.33	0.38
50	0.64 $\pm$ 0.02	0.63	0.59	0.51	0.45	0.50
75	0.82 $\pm$ 0.01	0.80	0.78	0.71	0.64	0.65

Latency. Per-email wall-clock breakdown on a 2023 MacBook (M2, 16 GB): SLM inference 587 ms, SBERT embedding 25 ms, HDBSCAN one-shot 6 ms, calibrator 0.4 ms, missing-context regex 0.8 ms; **end-to-end 614 ms / email**. The cloud upper bound (GPT-5.4 over HTTPS) is **4,993 ms / email**—8.5 $\times$ slower than the entire local pipeline.

7 Analysis

7.1 Ablations

Calibrator choice (Table 3). No single backend dominates. Calibrators that exploit the 76% `low` prior (unbalanced LogReg, RF, GBM) achieve the highest accuracy but their predicted distributions collapse toward $\{5, 3, 92\}$ and Spearman ρ is no better than the raw SLM. Class-balanced LogReg is the cleanest win: largest macro-F1 lift while maintaining the SLM’s ability to rank emails by urgency, with predicted distribution $\{23, 40, 37\}$ closer to ground-truth shape than any other backend.

Active-learning sampling strategy (Table 4). Random sampling matches or slightly exceeds entropy-based AL at every budget tested. The proximate cause is mechanical: the SLM’s three-bin confidence string maps 100 emails onto only $3 \times 3 = 9$ distinct (label, conf) buckets. Entropy AL spends its first labels exclusively in the highest-entropy bucket (predicted `normal` with low confidence), which happens to be the rare class where the model has the least to learn. Random sampling naturally surveys the `low` majority where the model makes most of its over-prediction errors. The implication for production is clear: until the serving infrastructure exposes real per-class log-probs (Ollama does not as of writing), *cluster-stratified random sampling* is a defensible default; our `entropy_diverse` variant achieves middle-ground ρ and is the production fallback we recommend.

Cluster refinement. Running the full calibrated pipeline with vs. without HDBSCAN refinement: accuracy moves from 0.43 to 0.46 (+0.03) and macro-F1 from 0.39 to 0.40 (+0.01) on the held-out fold. The lift is small in absolute terms but always positive across folds and concentrated on seven outlier-snap cases (single `urgent` predictions inside otherwise-coherent `low` clusters); we retain refinement because it is essentially free and protects against the most user-visible mistakes.

7.2 Error analysis

We sampled 12 mis-classifications from the post-calibration LogReg-balanced predictions and categorised them.

Table 5: Error categorisation. Over-prediction of `urgent` on syntactically polite requests dominates the residual macro-F1 gap to GPT-5.4.

Error type	Count	Example (SLM $\rightarrow$ GT)
Over-prediction of <code>urgent</code> on soft requests	5	“Could you take a look when you get a chance?” (<code>urgent</code> $\rightarrow$ <code>normal</code>)
Newsletter / FYI mis-labelled <code>urgent</code>	3	Bloomberg market wrap with “deadline” in boilerplate (<code>urgent</code> $\rightarrow$ <code>low</code>)
Genuinely missing-context error	2	“Per our call, please send the file ASAP” (<code>low</code> $\rightarrow$ <code>urgent</code>)
Rare-class boundary (<code>urgent</code> / <code>normal</code>)	2	Internal coordination request, no explicit deadline (<code>urgent</code> $\rightarrow$ <code>normal</code>)

The dominant residual error is over-prediction of urgency on syntactically polite requests. The few-shot prompt and post-hoc calibrator together cut these from 40/100 to 23/100 but do not eliminate them. The two genuinely missing-context errors are exactly the cases the regex detectors target—if we had used the missing-context flag as a tiebreaker into a manual queue, we would have correctly deferred both. The rare-class boundary errors are the `urgent/normal` distinction we already saw at the inter-annotator-agreement stage: two of three annotators disagreed on these in the original labelling, so we should not expect a model to do better than the labelling itself. We did not observe any case where the post-hoc calibrator pushed a *correct* SLM prediction over the boundary; calibrated mistakes were all already mistakes in the raw SLM.

8 Limitations and Broader Impact

The 100-email pilot is too small for confident per-class metrics: the rare classes (`urgent=11`, `normal=13`) yield recall standard errors ≥ 0.15 that we cannot close without more annotated data. The proxy oracle for missing-context yields a single positive on the full pilot and cannot rank detectors against each other; the human-annotated 35-email subset (§5) gives a usable signal but is itself small. Our entropy-AL conclusion is conditional on Ollama not exposing log-probs; a backend with real per-class probabilities (e.g., `llama.cpp` with the `logprobs` extension) may restore the expected ordering between random and entropy sampling.

Broader impact & ethics. The system is designed to keep email content on-device, which is a positive privacy posture, but on-device ML still carries risks. (i) The 5-detector missing-context heuristic produces a high-rate “needs context” signal that, if treated as authoritative, could nudge users to dismiss legitimately urgent mail—we therefore surface it as *instrumentation* (visible evidence spans) rather than as a hard classifier. (ii) Personalisation via demonstration injection learns from the user’s labelling biases; a user who systematically under-prioritises a class of senders may have those biases amplified by the prompt manager. (iii) The Enron pilot is from a corporate context in the early 2000s and likely under-represents modern conversational patterns (emoji, informal pings); production use should re-validate on a contemporary corpus.

9 Conclusion

We built a fully on-device email triage pipeline that addresses the three required functionalities of CS 288 Topic 18 with a 1.5B Qwen2.5-Instruct model, post-hoc calibration, HDBSCAN-based cluster refinement, and a 5-detector regex heuristic for missing-context, plus an interactive active-learning CLI with explicit prompt-history compression. On a 100-email Enron pilot we report macro-F1 0.39 (vs. 0.28 raw, 0.58 cloud upper bound), Spearman ρ 0.51 at 50 user corrections, $8.5\times$ lower per-email latency than the cloud baseline, and zero email content uploaded. Our most actionable finding is the negative one: random sampling matches entropy AL on this dataset because the SLM’s coarse confidence string collapses 100 emails onto nine buckets, motivating cluster-stratified random sampling as the responsible default until local backends expose real log-probs.

A Implementation Details

Repository layout. The implementation is split across the following modules.

- `paths.py`: centralised configuration.
- `slm_scorer_v2.py`: calibrated few-shot prompt (§4.1).
- `calibration.py`: 13-feature post-hoc calibrators (§4.1).
- `clustering_refinement.py`: SBERT embeddings + HDBSCAN (§4.2).
- `missing_context_detector.py`: 5 regex detectors (§5).
- `prompt_manager.py`: prompt-history compression (§4.3).
- `al_interactive.py`: interactive CLI (§4.3).
- `evaluate_extended.py`: unified evaluation harness.
- `feedback_efficiency.py`: random-vs-entropy ablation (Table 4).
- `demo.html`: single-file privacy-first inbox UI.

Calibrated few-shot SLM prompt (excerpt). The system prompt is fixed; the user prompt is templated per email. Demos are sampled to match the empirical 1:1:8 ratio.

```
SYSTEM: You score emails for urgency. Labels: urgent / normal / low.
Most emails are LOW urgency. In a typical inbox, ~75% are LOW,
~15% NORMAL, ~10% URGENT. Output JSON only:
{"urgency": ..., "urgency_score": 0-100, "deadline": "today" |
  "this week" | "no reply needed", "signals": [...],
  "missing_context": true/false, "reason": ...}
```

```
USER: Subject: <subject>
      From:    <sender>
      Body:    <first 800 chars of body>
Output:
```

Reproducibility. All experiments are CPU-only and deterministic modulo the AL random seeds (0–4). Running `evaluate_extended.py` reproduces Table 3 and Table 4 from the cached SLM and GPT-5.4 outputs in <30 s without re-querying any model.

Contribution Statement.

- **Hansheng Zeng** built the original Stage-2 clustering refinement and the missing-context detector module, set up the Qwen2.5-1.5B + Ollama serving stack, produced the GPT-5.4 cloud-baseline comparisons, and wrote the introduction, related-work, and method-overview sections of this report.
- **Jiaming Zhao** designed the post-hoc calibration framework, implemented the calibrated few-shot scorer, the calibration module, the prompt manager, the interactive AL CLI, and the unified evaluation harness, ran the feedback-efficiency study, and built the privacy-first `demo.html`.
- **JJ Sandhu** manually labelled 50 of the 100 emails, ran inter-annotator agreement, and wrote the dataset and qualitative-error sections.
- All authors jointly wrote and reviewed the report.

GenAI Statement. We used Claude (Anthropic, via `claude.ai`) as a consulting tool for: (1) clarifying debugging errors and unfamiliar library behavior in the calibration pipeline, (2) explaining LaTeX syntax for tables/figures and formatting boilerplate, and (3) high-level workflow planning (e.g., general steps for evaluation pipelines). No assignment text, dataset identifiers, or task-specific prompts were pasted into Claude. Claude was not used to generate analysis, interpret our results, annotate data, or produce solutions/code for any assignment part. All code, analysis, and written content were authored, reviewed, and understood by us. The 100-email ground-truth dataset was annotated entirely by humans without AI assistance.

References

- Douglas Aberdeen, Ondrej Pacovsky, and Andrew Slater. The learning behind Gmail Priority Inbox. In *LCCC: NIPS 2010 Workshop on Learning on Cores, Clusters, and Clouds*, 2010.
- Andi Ahmeti and Permiso Security. Cross-prompt injection attacks on AI email summarization assistants. <https://permiso.io/blog/copilot-prompt-injection-ai-email-phishing>, 2026.
- Apple Machine Learning Research. Introducing Apple’s on-device and server foundation models. <https://machinelearning.apple.com/research/introducing-apple-foundation-models>, 2024.
- Ron Bekkerman, Andrew McCallum, and Gary Huang. Automatic categorization of email into folders: Benchmark experiments on Enron and SRI corpora. Technical Report IR-418, University of Massachusetts Amherst, 2005.
- Laura A. Dabbish, Robert E. Kraut, Susan Fussell, and Sara Kiesler. Understanding email use: Predicting action on a message. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 691–700. ACM, 2005.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yarin Gal, Riashat Islam, and Zoubin Ghahramani. Deep bayesian active learning with image data. In *International Conference on Machine Learning (ICML)*, pages 1183–1192, 2017.
- Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*, 2011.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Bryan Klimt and Yiming Yang. The Enron corpus: A new dataset for email classification research. In *European Conference on Machine Learning (ECML)*, volume 3201 of *Lecture Notes in Computer Science*, pages 217–226. Springer, 2004.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024.
- Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*, 2024.
- Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017.
- Microsoft. Data, privacy, and security for Microsoft 365 Copilot. <https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-privacy>, 2025.
- Ollama. Ollama: Get up and running with large language models locally. <https://ollama.com/>, 2026.
- OpenAI. GPT-5.4 model documentation. <https://platform.openai.com/docs/models>, 2026.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7957–7968, 2023.
- Qwen Team. Qwen2.5: A party of foundation models. <https://qwenlm.github.io/blog/qwen2.5/>, 2024.

- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.
- Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. A survey of deep active learning. *ACM Computing Surveys*, 54(9):1–40, 2021.
- Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1):21–41, 2002.
- Burr Settles. Active learning literature survey. Technical Report 1648, University of Wisconsin–Madison Department of Computer Sciences, 2009.
- Steve Whittaker and Candace L. Sidner. Email overload: Exploring personal information management of email. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 276–283. ACM, 1996.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations (ICLR)*, 2023.